

# Config Check

LLM-assisted configuration anomaly detection for Junos networks

Proof of concept · Proxmox + EVE-NG lab · November 2026

# What the tool does

Reads every device's running config, compares it with how the network is meant to be built, and rates every deviation 1–5 on one dashboard.



## Detect

Exact template rules per role: base, infrastructure and service config. Plus drift between devices in the same role.



## Explain

A local LLM reads the design docs and reviews each config section and every change, with strict grounding checks.



## Track

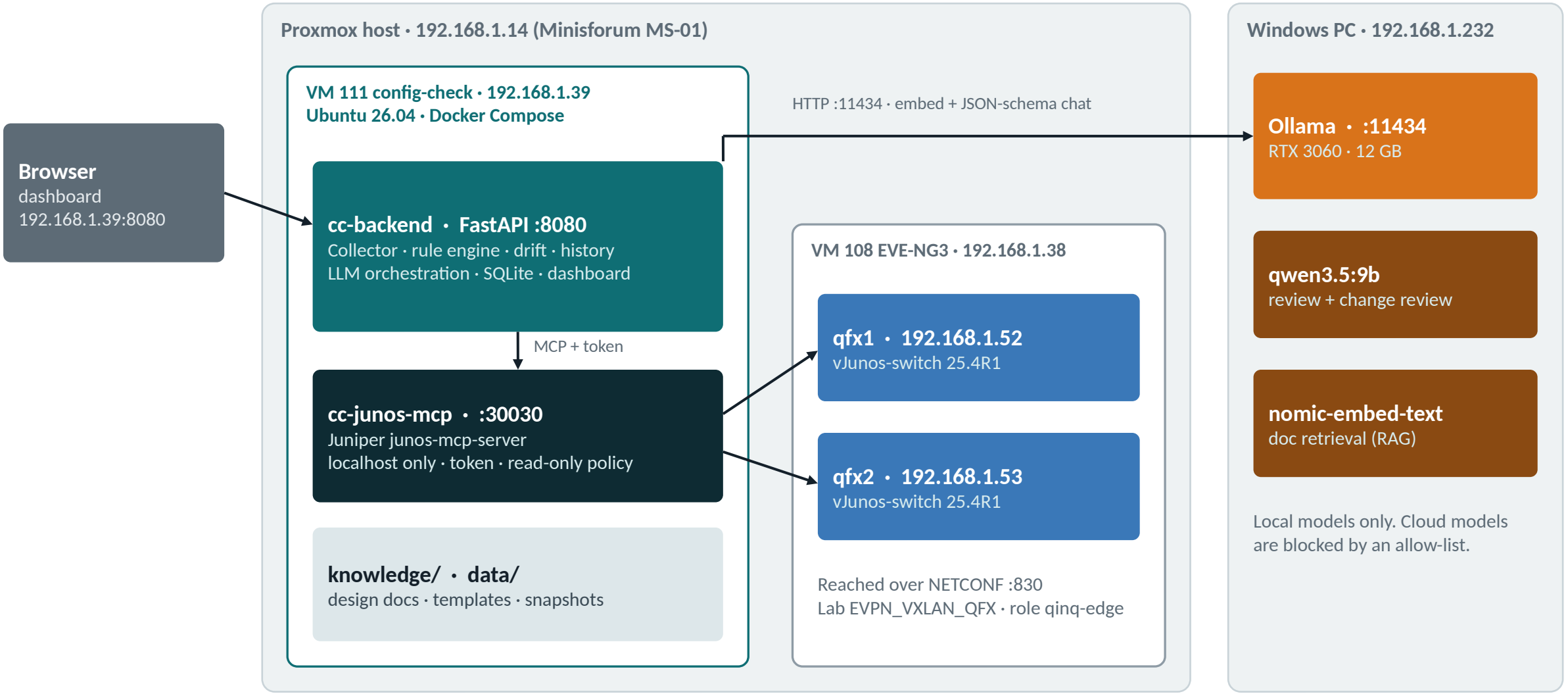
Every scan is a config snapshot. Diffs, Junos commit history, regressions and pinned baselines.



## Prioritise

Severity, compliance score, accepted exceptions and a suggested Junos fix for each finding.

# Architecture



# Components

| Component | Runs on | Address | Job |
|---|---|---|---|
| **Dashboard + API** | cc-backend container | 192.168.1.39:8080 | UI, scan control, findings, changes, decisions |
| **Scan engine** | cc-backend container | n/a | Collect, rules, drift, diff, LLM calls, scoring |
| **Junos MCP server** | cc-junos-mcp container | 127.0.0.1:30030 | Device access over NETCONF, block-listed to read-only |
| **Store** | SQLite + files on the VM | /opt/config-check/data | Scans, findings, changes, commits, baselines, snapshots |
| **Knowledge** | Files on the VM | /opt/config-check/knowledge | HLD, base / infra / service templates |
| **LLM runtime** | Ollama on Windows PC | 192.168.1.232:11434 | qwen3.5:9b (review), nomic-embed-text (retrieval) |
| **Lab devices** | EVE-NG VM 108 | 192.168.1.52 / .53 | 2 × vJunos-switch 25.4R1, role qinq-edge |
| **Hypervisor** | Proxmox VE 9 | 192.168.1.14 | Hosts the config-check and EVE-NG VMs |

# What happens in a scan




### 1 Collect

Config, facts and commit history for each device, via MCP

Code

### 2 Rules

Each role's templates checked line by line, scoped per VR and interface

Code

### 3 Drift

Lines on one device in a role that its peers don't have

Code

### 4 History

Diff against the last snapshot or a pinned baseline, plus risk hints

Code

### 5 LLM review

Changes, then each config section, checked against the retrieved design text

LLM

### 6 Guards

Drop anything ungrounded, duplicated, no-op or self-contradicting

Code

### 7 Store + show

Findings, states (new / recurring / regressed / accepted), score

Code

About 3 minutes for 2 devices with the model warm: collection about 20 s, rules under 1 s, LLM about 25 s per config section.

# What the tool checks against

## Design docs

`knowledge/design/*.md`

Prose HLD/LLD. Split by heading, embedded, and retrieved per config section for the LLM.

## Templates

`knowledge/templates/{base,infra,services}`

Junos set lines grouped into rules with ID, severity and design reference. Checked exactly by code.

## Inventory

`config/inventory.yaml`

Roles, the devices in each, which templates apply, per-role variables and drift-ignore patterns.

```
templates/services/sipt-vr.set

# @id SVC-BGP-AUTH @sev 4
#   @title "eBGP session has no auth"
#   @doc "HLD-qinq-edge §4.2"
# @scope ^set routing-instances
#   (?P<vr>SIPT\S*) protocols bgp
#   group CISCO-ASR\s
set routing-instances {{vr}} protocols
  bgp group CISCO-ASR authentication-key {{*}}

# forbidden line: prefix with !
!set system services telnet{{*}}
```

# How the LLM is used

## Retrieve

nomic-embed-text indexes 40 passages: HLD sections plus template rules. The 4 closest are sent with each config section.

## Review

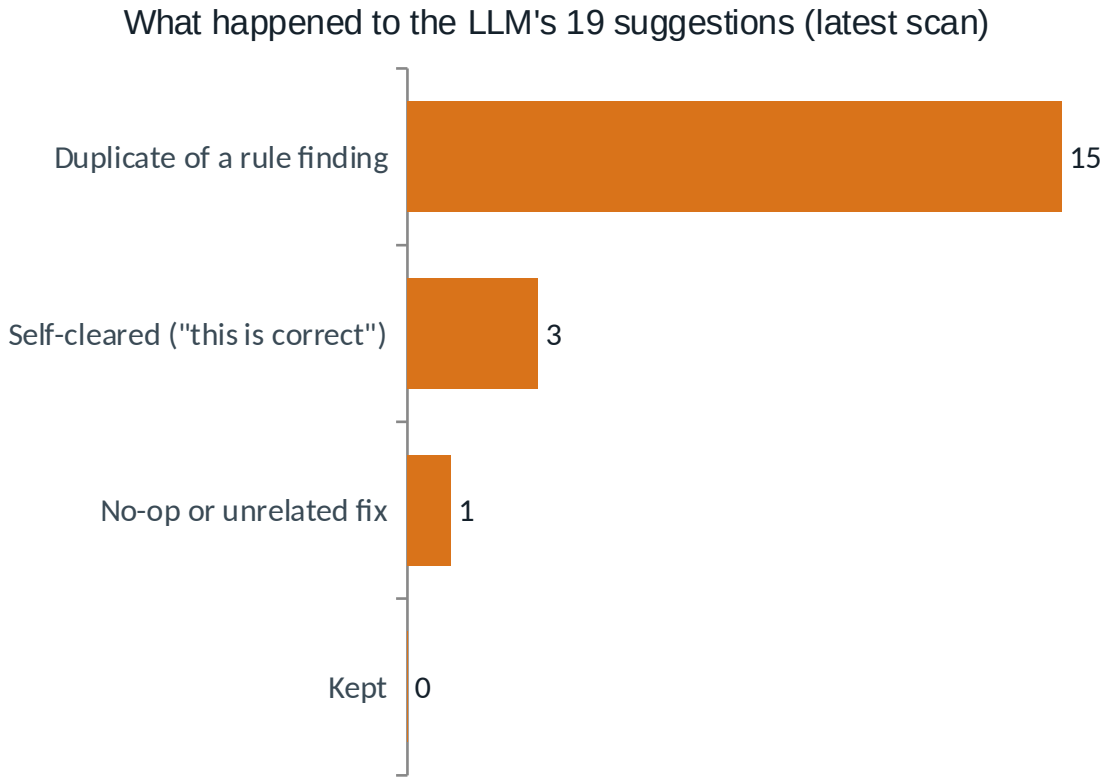
qwen3.5:9b gets one config section at a time (system, interfaces, routing), plus what the rules already found.

## Change review

Each diff is summarised with an intent and a 1–5 risk. Code risk hints set the floor.

## Guards

Every LLM claim must quote real config lines and a design sentence word for word, or it's dropped.




*The guards work: nothing wrong reached the dashboard. But at 9B the model finds nothing the rules don't already catch.*

# Code vs LLM: who does what

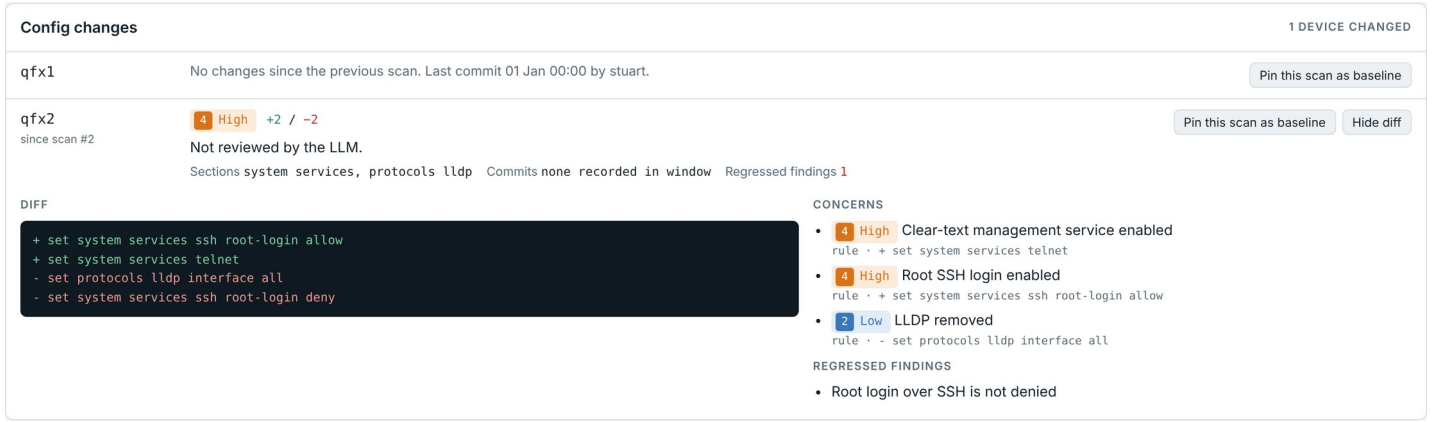
## Code (deterministic)

- Template compliance: exact, repeatable, instant
- Scoped checks per VR, interface or VLAN
- Drift between devices in the same role
- Config diffs, commit window, regressions
- Risk hints for known-dangerous changes
- Fact-checking every LLM claim

## LLM (judgement)

- Reads the prose design docs
- Plain-English summary and intent for each change
- Spots issues that span sections, where templates can't
- Best next uses: draft template rules from the HLD and explain findings for the NOC
- Limited by a 9B model on a 12 GB GPU

# Scans as backups: change tracking




*Lab simulation: qfx2 enabled telnet, removed LLDP and reverted a root-login fix*

### Finding states

**NEW** first time seen **RECURRING** also in last scan

**REGRESSED** gone last scan, back now **ACCEPTED** exception, kept across scans

## Snapshots

Every scan stores each device's config. The last 50 scans are kept, and pinned baselines are never deleted.

## Diff + commits

Compared with the previous scan, with show system commit for who, when and the comment.

## Risk

17 code hints (telnet, root SSH, BGP auth/policy removed…) plus the LLM summary.

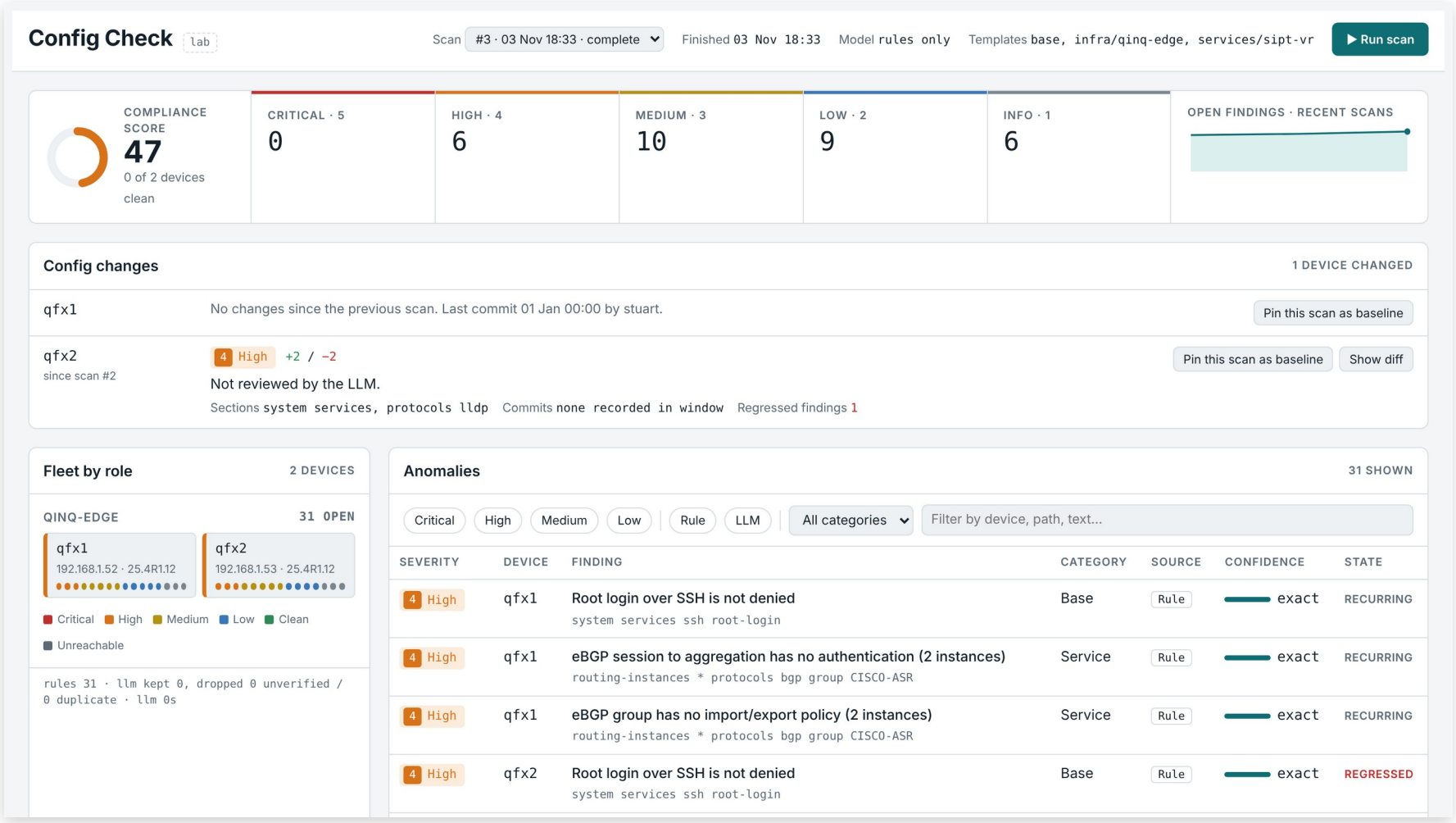
## Regressions

A finding that was fixed and comes back is flagged REGRESSED.

## Baselines

Pin a known-good scan per device and see drift from it on every scan.

# The dashboard




*Lab simulation data*

## Summary

Score, counts by severity, trend

## Config changes

Diff, risk, commits, pin baseline

## Fleet by role

Device tiles coloured by worst finding

## Anomalies

Filter by severity, source, category

## Detail

Evidence, design ref, Junos fix, accept

# Read-only by design: three layers

## 1 · Device

`Junos login class cc-ro`

permissions view + view-configuration; request / clear / restart / file / configure denied. SSH key only.

## 2 · MCP server

`block.cfg + block.cmd`

Every config load blocked; only show commands allowed. Bound to localhost with bearer-token auth.

## 3 · Backend

`Tool allow-list`

Calls only get_junos_config, gather_device_facts and show commands. Write tools are never called.

Data never leaves the lab: local Ollama only (cloud models refused by an allow-list). Secrets are masked by Junos in configs, and keys and tokens are stored with mode 600.

# PoC results on the lab

**2**
vJunos leaves scanned

**23**
template rules

**28**
real findings, 6 High

**~3 min**
per scan, model warm

### What it found on the real leaves

- eBGP to the Cisco ASR has no authentication and no import/export policy (both VRs, both leaves)
- Root SSH login allowed; no NTP, so devices run on local clock; no remote syslog
- VLAN QINQ-117-111 carries outer tag 107: the name doesn't match the service
- No BFD or prefix limits; leftover MC-LAG config; IPv6 RA on fxp0; syslog drift between the leaves

# Next steps

## Real standards

Replace the starter HLD and templates with production design docs.

## LLM where it adds value

Draft template rules from the HLD for review; explain findings for the NOC.

## Bigger model trial

Try gpt-oss:20b for cross-section and cross-device reasoning.

## More roles and devices

PE, P and RR roles; MX and PTX; scheduled scans with alerts.